

Netflix – Data Engineer Interview Q&A

Q1. Design a scalable schema for tracking real-time viewing data across millions of users.

A1. A scalable schema typically follows a **fact-dimension model**. The **fact table** stores events such as view_id, user_id, content_id, timestamp, device_id, watch_duration.

Dimensions include users, content, devices, and time. For scalability:

- Use partitioning (e.g., by date, region).
- Store raw events in Parquet/Delta Lake for analytics.
- Stream ingestion via Kafka + Spark.

This schema supports both real-time and batch analytics at Netflix scale.

Q2. Explain the differences between star and snowflake schemas.

A2.

- **Star Schema:** Dimensions are denormalized (flat tables connected to a central fact). Faster queries, simple design.
 - **Snowflake Schema:** Dimensions are normalized into multiple related tables. Saves storage but requires more joins, so queries may be slower. Netflix typically uses **star schemas** for BI tools due to query efficiency at scale.
-

Q3. Define a data governance framework and its importance.

A3. A data governance framework ensures data is **secure, accurate, and compliant**.

Components:

- **Data ownership & stewardship.**
- **Data quality checks.**
- **Access control & audit logs.**
- **Compliance** (GDPR, CCPA).

For Netflix, governance ensures content usage, user data, and ad targeting comply with privacy standards while maintaining trust and accuracy.

Q4. Describe an ETL pipeline for ingesting and processing user viewing data at Netflix scale.

A4.

1. **Ingestion:** Kafka streams capture user activity (play, pause, search).

2. **Storage:** Data stored in S3/ADLS in raw zone.
 3. **Transformation:** Spark/Presto jobs aggregate session data.
 4. **Loading:** Curated data stored in Redshift/BigQuery/Delta Lake.
 5. **Consumption:** Power BI/Looker dashboards for analytics.
This hybrid batch + streaming pipeline ensures both real-time monitoring and long-term analytics.
-

Q5. How would you automate data quality checks in streaming data workflows?

A5.

- Use **schema validation** with Avro/Protobuf.
 - Apply **data profiling rules** (null checks, ranges).
 - Implement **real-time anomaly detection** with Spark Structured Streaming.
 - Route invalid events to a **dead-letter queue (DLQ)**.
 - Automate alerts with CloudWatch/Datadog.
This ensures reliable streaming pipelines.
-

Q6. Write SQL to find the most frequently rented (or watched) items.

A6.

```
SELECT content_id, COUNT(*) AS watch_count
FROM viewing_events
GROUP BY content_id
ORDER BY watch_count DESC
LIMIT 10;
```

This retrieves the top 10 most watched items.

Q7. Write SQL to list users who joined within a specific timeframe.

A7.

```
SELECT user_id, name, signup_date
FROM users
WHERE signup_date BETWEEN '2024-01-01' AND '2024-12-31';
```

This filters users based on their signup date.

Q8. Design a system to track movie counts per genre annually.

A8.

- Store **fact table** with view_id, user_id, genre_id, watch_time, year.
- Aggregate using Spark/SQL:

```
SELECT genre, YEAR(timestamp) AS year, COUNT(DISTINCT content_id) AS  
movies_watched
```

```
FROM viewing_events v
```

```
JOIN content c ON v.content_id = c.id
```

```
GROUP BY genre, YEAR(timestamp);
```

This supports annual genre-based reports.

Q9. Explain the architecture of Hadoop and the MapReduce paradigm.

A9.

- **HDFS** stores large datasets across clusters with replication.
 - **MapReduce** splits tasks into **Map** (parallel processing) and **Reduce** (aggregation).
E.g., counting views per movie → Map emits (movie_id, 1), Reduce sums counts.
Though Spark replaced MapReduce at Netflix, Hadoop concepts still apply to legacy systems.
-

Q10. Discuss batch vs. stream processing; when would you prefer one over the other?

A10.

- **Batch:** Processes large data chunks at intervals (daily summaries, reports).
 - **Streaming:** Processes data in near real-time (live recommendations, fraud detection).
Netflix uses streaming for **real-time engagement** (like "Trending Now") and batch for **long-term insights** (monthly churn analysis).
-

Q11. How would you design a real-time recommendation engine?

A11.

- Ingest events via **Kafka**.
 - Process with **Spark Streaming/Flink** to update user embeddings.
 - Use **collaborative filtering (ALS)** or **content-based models** for recommendations.
 - Store in Redis for low-latency retrieval.
This enables Netflix to suggest content instantly after user interaction.
-

Q12. Compare Spark and Hadoop MapReduce; when is each appropriate?

A12.

- **Hadoop MapReduce**: Disk-based, slower, fault-tolerant. Good for large batch jobs with simple transformations.
 - **Spark**: In-memory, faster, supports streaming + ML. Best for iterative processing and real-time workloads.
Netflix primarily uses Spark for performance and versatility.
-

Q13. Outline ETL pipeline best practices—fault tolerance, monitoring.

A13.

- Implement **idempotent loads**.
 - Use **checkpointing** for streaming jobs.
 - Automate retries and backfills.
 - Monitor with **metrics dashboards** (Prometheus, Grafana).
 - Ensure **data lineage & audit logging**.
These ensure pipelines run reliably at scale.
-

Q14. How would you implement a recommendation engine pipeline (use case example)?

A14. Example: Movie recommendation pipeline.

- Ingest watch events via Kafka.
- Train ML models (collaborative filtering) on Spark MLlib.

- Update real-time feature store with embeddings.
 - Deliver recommendations through APIs.
This enables personalized recommendations across devices.
-

Q15. Optimize SQL queries operating at petabyte scale.

A15.

- Use **partitioning and clustering**.
 - Apply **filter pushdown**.
 - Avoid SELECT *; fetch only needed columns.
 - Use **materialized views**.
 - Leverage **columnar formats** like Parquet.
This reduces I/O and query latency significantly.
-

Q16. Describe an algorithm for reservoir sampling on data streams.

A16. Reservoir sampling selects **k random samples** from a stream of unknown length.

Algorithm:

1. Fill reservoir with first k items.
 2. For each new item i ($i > k$), generate random $r \in [0, i]$.
 3. If $r < k$, replace reservoir[r] with item i.
This ensures uniform sampling.
-

Q17. Implement an in-place matrix rotation algorithm ($N \times N$, 90° clockwise).

A17.

```
def rotate(matrix):
```

```
    n = len(matrix)
```

```
    for i in range(n):
```

```
        for j in range(i, n):
```

```
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
```

```
    for row in matrix:
```

```
        row.reverse()
```

return matrix

This first transposes, then reverses rows.

Q18. Build a priority queue using a singly-linked list and explain trade-offs.

A18. Insert nodes in sorted order by priority. Operations:

- **Enqueue:** $O(n)$ (traverse to correct position).
- **Dequeue:** $O(1)$ (remove head).

Trade-off: Simple implementation, but insertion costlier than heap-based PQ ($O(\log n)$).

Q19. Write an SQL query to list the top 3 salaries per department, handling ties.

A19.

```
SELECT department, employee, salary
```

```
FROM (
```

```
    SELECT department, employee, salary,
```

```
        DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) as rnk
```

```
    FROM employees
```

```
) t
```

```
WHERE rnk <= 3;
```

This ensures ties are included.

Q20. Design a system to prevent account sharing without compromising user privacy.

A20.

- Monitor unusual patterns (multiple logins from distant geolocations).
 - Apply device fingerprinting and session limits.
 - Use ML anomaly detection for account misuse.
 - Notify users when suspicious activity is detected.
Privacy preserved by avoiding direct identity tracking.
-

Q21. Propose an ad delivery system integrated with Netflix's recommendation engine.

A21.

- Use user profiles and embeddings from recommendation system.
 - Select ads relevant to viewing history.
 - Run **real-time bidding (RTB)** integrated with recommendation pipelines.
 - Ensure frequency capping and user privacy.
This delivers personalized ads while maintaining engagement.
-

Q22. Plan an A/B test for optimizing Netflix's signup funnel flows.

A22.

- Define KPIs (conversion rate, churn reduction).
 - Randomly assign users to control vs variant groups.
 - Ensure **statistical significance** with sample size calculation.
 - Monitor using analytics dashboards.
This validates UX improvements scientifically.
-

Q23. Discuss Netflix's culture and leadership values relevant to a Data Engineer.

A23. Netflix emphasizes **freedom with responsibility**, **data-driven decision making**, and **candor**. For a DE, this means:

- Owning pipelines end-to-end.
 - Ensuring data quality and transparency.
 - Collaborating across teams to maximize impact.
Aligning with culture is as important as technical skills.
-

Q24. Explain a streaming pipeline with Kafka, Spark, and HDFS architecture.

A24.

- Kafka ingests real-time events.
- Spark Streaming consumes Kafka topics, processes transformations.
- Data written to HDFS/Delta Lake.

- Downstream systems (Presto/Redshift) query processed data.
This is a standard Netflix-style real-time ETL architecture.
-

Q25. Solve: given movie budgets and total cap K, find optimal budget cap per show.

A25. This is a **binary search problem**. Sort budgets, apply binary search on cap value. For each mid cap, sum $\min(\text{budget}, \text{cap})$. If $\text{sum} \leq K$, adjust search. Returns optimal cap ensuring total $\leq K$.

Q26. Recommend 3 unseen movies based on recent favorite genres.

A26.

- Identify top genres watched in last N sessions.
- Query unseen movies in same genres.
- Rank by popularity or rating.

```
SELECT movie_id
```

```
FROM movies
```

```
WHERE genre IN (SELECT genre FROM user_recent_genres)
```

```
AND movie_id NOT IN (SELECT movie_id FROM user_watched)
```

```
ORDER BY rating DESC
```

```
LIMIT 3;
```

Q27. Reverse-engineer a production Spark job from query or behavior description.

A27. Steps:

- Analyze input/output datasets.
 - Trace job lineage via Spark logs/UI.
 - Identify transformations (joins, aggregations).
 - Reconstruct pipeline logic in PySpark.
This skill is tested to gauge debugging and comprehension.
-

Q28. Explain rank() functions and nuances in SQL.

A28.

- **RANK():** Skips ranks on ties (1, 2, 2, 4).
 - **DENSE_RANK():** No gaps (1, 2, 2, 3).
 - **ROW_NUMBER():** Unique sequential numbers.
Use case depends on whether ties should produce gaps.
-

Q29. Handle Spark or Flink pipeline failure troubleshooting.

A29.

- Check logs (executor, driver, task).
 - Enable checkpointing for recovery.
 - Review resource allocation (memory, cores).
 - Handle skew with repartitioning.
 - Test smaller batches for reproducibility.
- Systematic debugging ensures stable pipelines.
-

Q30. Share how you approach behavioral storytelling during interviews (e.g., fixing mistakes).

A30. Use **STAR method** (Situation, Task, Action, Result). Example: “A data pipeline failed in production due to schema drift. I quickly added schema validation, implemented DLQ handling, and reprocessed missed data. Result: Reduced downtime from hours to 30 minutes.” This shows ownership and problem-solving.

Meta – Data Engineer Interview Q&A

Q1. Design a dashboard metric: unique login counts per user.

A1. The metric tracks distinct users logging in daily. SQL:

```
SELECT login_date, COUNT(DISTINCT user_id) AS unique_logins
FROM logins
GROUP BY login_date;
```

For the dashboard:

- Use daily/weekly granularity.
- Partition by region/device for deeper analysis.
- Store in a summary table for fast BI visualization.

Q2. Propose a KPI to analyze feature impact in a News Feed.

A2. Possible KPIs:

- **Engagement rate** (clicks, likes, shares per impression).
- **Dwell time** on feed posts.
- **Session length** per user.

Measure before vs. after feature rollout using A/B tests. This identifies if the feature increases engagement without harming retention.

Q3. Calculate video popularity on Facebook—formulation and query.

A3. Popularity can be defined as a weighted score of **views + likes + shares**. Example:

```
SELECT video_id,
```

```
    SUM(views) + 2*SUM(likes) + 3*SUM(shares) AS popularity_score
```

```
FROM video_events
```

```
GROUP BY video_id
```

```
ORDER BY popularity_score DESC;
```

This ranks videos by combined engagement.

Q4. Design a product experiment to measure engagement on a feature.

A4. Steps:

1. Define hypothesis (e.g., "New comment UI increases engagement").
 2. Identify metrics (comments per user, time spent).
 3. Split users into control vs treatment groups randomly.
 4. Run experiment for sufficient duration.
 5. Analyze results with t-tests/chi-square to ensure significance.
-

Q5. Design a data model/schema for a ride-sharing service.

A5.

- **Users:** user_id, name, rating.
- **Drivers:** driver_id, vehicle_id, rating.

- **Rides:** ride_id, user_id, driver_id, start_time, end_time, fare.
 - **Locations:** location_id, lat, long, city.
Partition rides table by date/region to handle scale.
-

Q6. Create DDL from an ERD containing many-to-many relationships.

A6. Example: Students ↔ Courses.

```
CREATE TABLE Students (student_id INT PRIMARY KEY, name VARCHAR(50));
```

```
CREATE TABLE Courses (course_id INT PRIMARY KEY, title VARCHAR(50));
```

```
CREATE TABLE StudentCourse (
```

```
    student_id INT,
```

```
    course_id INT,
```

```
    PRIMARY KEY(student_id, course_id),
```

```
    FOREIGN KEY(student_id) REFERENCES Students(student_id),
```

```
    FOREIGN KEY(course_id) REFERENCES Courses(course_id)
```

```
);
```

Q7. Model a system for global weather tracking data.

A7. Schema design:

- **Station:** station_id, location, country.
 - **WeatherReadings:** reading_id, station_id, timestamp, temperature, humidity, pressure.
 - Partition by timestamp (daily/hourly).
 - Store in columnar formats (Parquet) for queries.
Use Kafka + Spark Streaming for ingesting real-time readings.
-

Q8. Write SQL to find the second-highest salary in a table.

A8.

```
SELECT MAX(salary) AS second_highest
```

```
FROM employees
```

```
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Alternative (for handling ties):

```
SELECT salary
FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) t
WHERE rnk = 2;
```

Q9. Detect duplicates using SQL across a dataset.

A9.

```
SELECT user_id, COUNT(*)
FROM users
GROUP BY user_id
HAVING COUNT(*) > 1;
```

This identifies duplicate user records.

Q10. Count employees per department using aggregation.

A10.

```
SELECT department_id, COUNT(*) AS employee_count
FROM employees
GROUP BY department_id;
```

Q11. Join tables without common columns—how?

A11.

- Use **CROSS JOIN** for Cartesian product.
- Use **arbitrary join conditions** (e.g., time-based range). Example:

```
SELECT *
FROM orders o
JOIN promotions p
```

ON o.order_date BETWEEN p.start_date AND p.end_date;

Q12. Return top 3 earning employees per department via SQL.

A12.

SELECT department, employee, salary

FROM (

SELECT department, employee, salary,

DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk

FROM employees

) t

WHERE rnk <= 3;

Q13. Handle NULL values in arithmetic SQL queries.

A13. Use **COALESCE** to replace NULLs:

SELECT salary + COALESCE(bonus, 0) AS total_comp

FROM employees;

This prevents NULL propagation.

Q14. Use HAVING vs WHERE—give use cases.

A14.

- **WHERE** filters rows before aggregation.
- **HAVING** filters groups after aggregation.

Example:

SELECT department, COUNT(*)

FROM employees

GROUP BY department

HAVING COUNT(*) > 10;

Q15. Update text data to lowercase in SQL.

A15.

UPDATE users

SET email = LOWER(email);

Q16. Optimize a slow-running SQL query—best practices.

A16.

- Add **indexes** on join/filter columns.
 - Use **EXPLAIN plan** to identify bottlenecks.
 - Reduce subqueries, prefer **CTEs**.
 - Use **partitioning** on large tables.
 - Fetch only necessary columns.
-

Q17. Walk through SQL+Python paired coding interview (3 each).

A17.

- SQL: Joins, window functions, aggregations.
 - Python: DataFrame manipulations (pandas), parsing JSON, implementing algorithms (e.g., group by and aggregate).
- Interviews test both query logic and coding proficiency for ETL workflows.
-

Q18. Design a real-time streaming pipeline ensuring fault tolerance.

A18.

- Ingest with Kafka → Spark/Flink for transformations.
 - Use **checkpointing & WAL (write-ahead logs)**.
 - Store in Delta Lake/S3.
 - Use retries and DLQ for errors.
- This ensures exactly-once processing.
-

Q19. Describe a final interview loop: product sense + data modeling + SQL + coding.

A19. Meta's final loop tests:

- **Product sense:** Define right metrics.
 - **Data modeling:** Schema design.
 - **SQL:** Write complex queries.
 - **Coding:** Implement algorithms in Python.
Also includes behavioral evaluation on collaboration and impact.
-

Q20. Explain DE responsibilities and ownership mindset.

A20. A DE at Meta owns **end-to-end pipelines**: ingestion, transformation, quality, and delivery. Ownership means:

- Proactively improving systems.
 - Monitoring pipelines.
 - Partnering with stakeholders.
 - Ensuring reliability without needing constant supervision.
-

Q21. Expect medium-level Python and SQL coding; 25 mins each.

A21. Candidates should be prepared for:

- SQL: joins, window functions, case statements.
 - Python: list/dict operations, pandas transformations.
 - 25 minutes = concise, optimized solutions under time pressure.
-

Q22. Outline Data Engineer phone screen: SQL focus on window functions, joins.

A22. Expect:

- Write queries using ROW_NUMBER(), RANK().
 - Perform joins across multiple tables.
 - Handle NULLs and filtering.
- Interviewers check both correctness and efficiency.
-

Q23. Prepare with LeetCode medium SQL and Python (pandas, NumPy).

A23. Meta often mirrors LeetCode-style problems:

- SQL: aggregation + ranking queries.

- Python: array, hashmap, string manipulation.
 - Pandas: groupby, pivot, merge.
Practice ensures time efficiency.
-

Q24. Expect product sense, data modeling, coding, and behavioral loops.

A24. Product sense tests KPI selection.

Data modeling ensures schema design skills.

Coding + SQL tests execution ability.

Behavioral loops test alignment with Meta's culture (impact-driven, collaborative).

Q25. Focus on data transformations, SQL joins, pandas during prep.

A25. Examples:

- SQL: Inner/outer joins, aggregations.
 - Pandas: groupby().agg(), merges.
 - Real scenarios: clean datasets, deduplicate, compute metrics.
Strong ETL skills are core at Meta.
-

Q26. Use articulating thinking, asking clarifying questions as advantage.

A26. Example: When asked to compute "engagement," clarify whether to use likes only or likes + comments. This shows product sense and avoids wrong assumptions.

Thinking aloud demonstrates structured problem-solving.

Q27. Linear coding: array, hashmap, heaps; DSA basics appear in DE interviews.

A27. Examples:

- Array: sliding window for moving average.
 - Hashmap: count frequency of items.
 - Heap: find top-k items.
Though DE is not DSA-heavy, core concepts are tested.
-

Q28. Emphasize query optimization and script clarity under time pressure.

A28. Interviewers prefer **clear, maintainable code**. Example: Use **CTEs** in SQL instead of nested subqueries. Write Python code with meaningful variable names and modular functions.

Q29. Practice mediator functions: join logic, window, aggregation under time constraints.

A29. Example problems:

- Find top N per group with window functions.
- Compute rolling averages.
- Join multiple datasets efficiently.
This reflects Meta's data-heavy environment.

Q30. Show trade-off reasoning—clarity in decision-making during coding/design.

A30. Example: "I chose star schema instead of snowflake because queries are faster, though storage cost is slightly higher. For Netflix-scale analytics, performance outweighs storage." Trade-off reasoning demonstrates senior-level thinking.

Apple – Data Engineer Interview Q&A

Q1. Explain Monkey Patching with an example.

A1. Monkey patching means **modifying a class or module at runtime**. Example in Python:

```
class Calculator:
```

```
    def add(self, a, b):
```

```
        return a + b
```

```
def new_add(self, a, b):
```

```
    return a + b + 10 # changed logic
```

```
Calculator.add = new_add # monkey patching
```

This overrides the existing add method. It should be used carefully because it can lead to unpredictable behavior in large systems.

Q2. Compare Redshift vs PostgreSQL.

A2.

- **PostgreSQL:** OLTP database, row-based, supports transactional workloads.

- **Redshift:** OLAP, columnar storage, optimized for large-scale analytical queries. Redshift is built for **data warehousing and big queries**, whereas PostgreSQL is best for **real-time, transactional workloads**.
-

Q3. Explain Parquet vs CSV data formats.

A3.

- **CSV:** Row-based, human-readable, no compression by default, inefficient for large analytical workloads.
 - **Parquet:** Columnar, supports compression, schema evolution, and efficient querying.
Apple would favor Parquet for **scalability and query performance** in analytics.
-

Q4. Contrast Spark and MapReduce processing models.

A4.

- **MapReduce:** Disk-based, slower, good for batch jobs.
 - **Spark:** In-memory, DAG execution, supports batch + streaming + ML.
Spark is faster and more flexible, which is why most modern Apple pipelines favor Spark.
-

Q5. Write Python code for binary tree traversal.

A5. Example: In-order traversal.

class Node:

```
def __init__(self, val):  
    self.val = val  
    self.left = None  
    self.right = None
```

def inorder(root):

```
    if root:  
        inorder(root.left)  
        print(root.val)
```

inorder(root.right)

Other traversals include preorder and postorder.

Q6. Optimize median-finding for large lists algorithmically.

A6. Instead of sorting ($O(n \log n)$), use:

- **Quickselect algorithm** → average $O(n)$.
 - **Heaps** → maintain max-heap of lower half and min-heap of upper half.
This approach is efficient for streaming median calculations.
-

Q7. Solve a mock “market basket analysis” schema design.

A7.

- **Transactions:** transaction_id, user_id, timestamp.
 - **Products:** product_id, category, price.
 - **Transaction_Items:** transaction_id, product_id.
This schema supports association rule mining (e.g., "users who bought iPhone also bought AirPods").
-

Q8. Design an end-to-end data pipeline for analytics.

A8.

1. Ingest logs/events from devices via Kafka.
 2. Store raw data in S3/ADLS.
 3. Transform using Spark (cleaning, deduplication).
 4. Load into Redshift/Snowflake.
 5. Expose via dashboards (Tableau/Looker).
Pipeline includes monitoring, retries, and CI/CD integration.
-

Q9. Write SQL using JOINS, window functions, and pivots.

A9. Example: Sales per month with running total.

SELECT month, product,

SUM(sales) AS monthly_sales,

SUM(SUM(sales)) OVER (PARTITION BY product ORDER BY month) AS running_total
FROM sales

GROUP BY month, product;

Pivoting depends on DB (e.g., PIVOT in SQL Server, CASE WHEN in standard SQL).

Q10. Explain optimizing database performance.

A10.

- Add indexes on frequently queried columns.
 - Use partitioning/sharding for large tables.
 - Avoid unnecessary subqueries.
 - Analyze execution plan to tune queries.
 - Use caching where applicable.
-

Q11. Walk through a full-day onsite with coding and modeling.

A11. At Apple, onsite loops include:

- SQL + Python coding challenges.
 - Data modeling exercises.
 - System design (ETL, pipelines).
 - Behavioral interviews testing ownership and problem-solving.
-

Q12. Discuss behavioral scenarios of missed deadlines.

A12. Example answer:

- Situation: Pipeline delivery delayed due to schema drift.
 - Action: Informed stakeholders, implemented hotfix, documented lessons.
 - Result: Reduced future delays via automation.
This shows accountability and problem-solving.
-

Q13. Write SQL to compute rolling aggregates for time-series.

A13.

```
SELECT user_id, event_date,
```

```
    SUM(events) OVER (PARTITION BY user_id ORDER BY event_date ROWS BETWEEN 6  
PRECEDING AND CURRENT ROW) AS rolling_weekly
```

```
FROM user_events;
```

Q14. Optimize ETL to minimize data latency.

A14.

- Use **micro-batching** or streaming instead of daily batch.
 - Parallelize transformations.
 - Apply CDC for incremental updates.
 - Tune pipeline concurrency and partitioning.
-

Q15. Explain recommendations for data storage for scale.

A15.

- Use **Parquet/ORC** for analytics.
 - Partition by time/region.
 - Apply data lifecycle policies (archiving cold data).
 - Store metadata in a catalog (Glue/Hive).
-

Q16. Design fault-tolerant pipelines in Spark/SQL.

A16.

- Enable **checkpointing** and WAL for streaming.
 - Build **idempotent jobs**.
 - Use retries with exponential backoff.
 - Route errors to DLQ.
-

Q17. Migrate pipelines from CSV to Parquet for performance.

A17. Steps:

1. Read CSV into Spark.

2. Clean/transform.
 3. Write to Parquet with partitioning.
 4. Update downstream queries to read from Parquet.
This reduces storage and improves query speed.
-

Q18. Setup CI/CD for data workflows in Apple's context.

A18.

- Version pipelines in Git.
 - Use Jenkins/Azure DevOps for build & test.
 - Automate deployments with Terraform/CloudFormation.
 - Test pipelines in staging before production.
-

Q19. Explain handling multi-region data ingestion.

A19.

- Deploy ingestion clusters regionally.
 - Use geo-distributed Kafka topics.
 - Replicate data asynchronously to central lake.
 - Apply GDPR-compliant residency rules.
-

Q20. Ensure data security and compliance in pipelines.

A20.

- Encrypt data at rest and in transit.
 - Tokenize or mask PII.
 - Use role-based access controls.
 - Enable audit logging.
 - Align with ISO/GDPR compliance.
-

Q21. Use logging for traceability in data pipelines.

A21.

- Structured logs with job ID, timestamp, input/output rows.
 - Send logs to centralized store (ELK stack).
 - Enable log-based alerts (failed jobs, anomalies).
-

Q22. Handle pipeline backpressure or delayed data ingestion.

A22.

- Use **Kafka with consumer lag monitoring**.
 - Apply **buffering** with queues.
 - Scale consumers dynamically.
 - Use backpressure mechanisms in Flink/Spark Streaming.
-

Q23. Explain joins across nested datasets efficiently.

A23. In Spark, use `explode()` to flatten arrays/maps before joining. Example:

```
df_exploded = df.withColumn("genre", explode(df.genres))
```

```
df_exploded.join(other_df, "genre")
```

Efficient flattening prevents shuffle overhead.

Q24. Troubleshoot slow queries in large databases.

A24.

- Check missing indexes.
 - Rewrite queries using CTEs.
 - Partition large tables.
 - Use caching/materialized views.
 - Monitor execution plans for hotspots.
-

Q25. Use Python to transform and pivot large datasets.

A25. Example with pandas:

```
import pandas as pd
```

```
df.pivot_table(index='user', columns='month', values='sales', aggfunc='sum')
```

This reshapes data for analysis.

Q26. Build a dashboard for monitoring ETL reliability.

A26. Metrics to include:

- Pipeline success/failure rate.
- Data latency (source to sink).
- Row count mismatches.
- Resource utilization.

Tools: Grafana, Tableau, or internal BI dashboards.

Q27. Handle merges and deduplication for changing schemas.

A27.

- Use **Delta Lake MERGE** with keys.
 - Apply deduplication with ROW_NUMBER().
 - Handle schema evolution with mergeSchema option.
-

Q28. Optimize Spark cluster configuration for workload.

A28.

- Set appropriate spark.sql.shuffle.partitions.
 - Choose executor memory/cores per workload.
 - Enable autoscaling clusters.
 - Use broadcast joins for small lookup tables.
-

Q29. Explain real-time vs batch decision-making for metrics.

A29.

- **Real-time:** Operational metrics (uptime, fraud detection, trending content).
 - **Batch:** Strategic metrics (monthly churn, long-term revenue trends).
- Apple often uses real-time monitoring + batch reporting together.
-

Q30. Align data solution design with Apple's culture of precision.

A30. Apple values **precision and reliability**. For a DE, this means:

- Designing fault-tolerant pipelines.
- Enforcing strict data quality rules.
- Documenting and testing rigorously.
- Ensuring accurate, trustworthy data products.

Microsoft – Data Engineer Interview Q&A

Q1. Explain end-to-end pipeline design for large-scale batch and streaming.

A1. A modern pipeline includes:

- **Ingestion:** Event Hubs/Kafka for streaming, ADF for batch.
 - **Storage:** ADLS Gen2 (raw zone, curated zone).
 - **Processing:** Spark on Synapse/Databricks for transformations.
 - **Serving:** Synapse SQL pools, Power BI dashboards.
 - **Monitoring:** Azure Monitor/Log Analytics for alerts.
- Design must ensure **scalability, fault tolerance, and data governance**.
-

Q2. How do you build fault-tolerant ETL systems in Azure?

A2.

- Enable **retry policies** in ADF activities.
 - Use **checkpointing** in Spark/Flink for streaming.
 - Design pipelines as **idempotent**.
 - Capture errors in **dead-letter queues**.
 - Implement monitoring + automated reruns.
- These ensure reliability in production.
-

Q3. Design a data warehouse using Synapse Analytics.

A3.

- Identify fact tables (sales, revenue).
- Identify dimensions (date, product, region).

- Use **hash distribution** for large joinable tables.
 - Partition fact tables by date for performance.
 - Expose data marts for BI reporting.
- Synapse's **MPP architecture** supports petabyte-scale workloads.
-

Q4. Write performant Spark code for data transformations.

A4. Best practices:

- Use DataFrame API instead of RDDs.
- Avoid UDFs; prefer built-in functions.
- Repartition on join keys.
- Cache intermediate results if reused.

```
df = df.filter(df.date >= '2024-01-01') \
    .groupBy("region") \
    .agg(F.sum("sales").alias("total_sales"))
```

Q5. Discuss data partitioning and indexing strategies in large tables.

A5.

- **Partitioning:** Improves query pruning; e.g., partition sales by date.
 - **Indexing:** Speeds up filtering; clustered indexes for range queries, non-clustered for lookups.
- Combine both to optimize query performance.
-

Q6. Optimize Python and SQL workflows under time constraints.

A6.

- In Python: Use **vectorized operations (NumPy, pandas)**.
 - In SQL: Avoid SELECT *, use **CTEs** for readability.
 - Precompute aggregates with materialized views.
 - Use parallelism in Spark or multiprocessing in Python.
-

Q7. Use decorator-based testing in data pipelines.

A7. In Python, decorators can enforce test conditions:

```
def test_non_empty(func):  
    def wrapper(*args, **kwargs):  
        result = func(*args, **kwargs)  
        assert result.count() > 0  
        return result  
    return wrapper
```

This helps validate pipeline outputs automatically.

Q8. Implement incremental loads in Azure Data Factory.

A8.

- Use **watermark columns** (LastModifiedDate).
 - Filter source using high-watermark.
 - Store checkpoint in metadata table.
 - Load only new/updated rows into sink.
This minimizes reprocessing and cost.
-

Q9. Manage credentials using managed identities vs service principals.

A9.

- **Managed Identity:** Automatically managed by Azure, no secrets needed.
 - **Service Principal:** Requires client ID/secret, more manual management.
Managed identities are preferred for security and simplicity.
-

Q10. Explain integrating Databricks with ADF for orchestration.

A10.

- Create **linked service** to Databricks in ADF.
- Add **Databricks Notebook activity** in pipeline.
- Pass parameters dynamically from ADF.
- Use ADF to orchestrate multiple notebooks across stages.

Q11. Handle schema change with minimal downtime.

A11.

- Implement **schema evolution** in Delta Lake (mergeSchema).
- Deploy changes via **blue-green deployment**.
- Add new columns as nullable for backward compatibility.
- Test schema updates in staging before production.

Q12. Secure PII data with row-level security in your pipeline.

A12.

- Use SQL **row-level security policies**.
- Mask or encrypt PII columns.
- Restrict access with RBAC.
- Store encryption keys in Key Vault.
This ensures GDPR/CCPA compliance.

Q13. Manage compute cost in cloud infrastructure (e.g., auto-scaling).

A13.

- Enable **autoscaling clusters** in Databricks.
- Use **spot instances** for non-critical workloads.
- Monitor resource utilization.
- Schedule shutdown of idle resources.
This optimizes cloud costs.

Q14. Write SQL queries for windowing and ranking in analytics.

A14. Example: Top 3 products per region.

SELECT region, product, sales

FROM (

SELECT region, product, sales,

RANK() OVER (PARTITION BY region ORDER BY sales DESC) AS rnk

FROM product_sales

) t

WHERE rnk <= 3;

Q15. Debug slow Spark jobs via event logs and Spark UI.

A15.

- Analyze DAG stages in Spark UI.
 - Check skewed partitions or shuffle size.
 - Inspect executor logs for memory errors.
 - Tune spark.sql.shuffle.partitions.
- These steps help locate bottlenecks.
-

Q16. Describe pipeline deployment via ARM templates or Terraform.

A16.

- Export ADF pipelines as **ARM templates**.
 - Deploy to other environments with parameter overrides.
 - Alternatively, use **Terraform** for infrastructure as code.
- This ensures consistent deployments.
-

Q17. Discuss troubleshooting data latency or missing data issues.

A17.

- Check source system lag.
 - Inspect pipeline retries or failures.
 - Validate watermark logic in incremental loads.
 - Monitor consumer lag in Kafka/Event Hubs.
-

Q18. Build unit tests for data transformations.

A18.

- Use **PyTest** for PySpark functions.
- Validate row counts, schema, and null handling.

- Create test datasets with edge cases.
This ensures correctness before deploying pipelines.
-

Q19. Explain monitoring and alerting using Azure Monitor.

A19.

- Send ADF/Databricks logs to Log Analytics.
 - Create metrics dashboards.
 - Configure alerts for failures, latency breaches.
 - Use Action Groups for notifications (email, Teams).
-

Q20. Describe a DR strategy for cloud-hosted data systems.

A20.

- Replicate data across regions.
 - Automate failover of compute clusters.
 - Backup metadata/catalogs.
 - Define RPO/RTO requirements.
This ensures high availability during outages.
-

Q21. Outline metadata management and cataloging (Purview, Unity Catalog).

A21.

- **Azure Purview:** Enterprise-wide catalog, lineage tracking.
 - **Unity Catalog:** Fine-grained access control in Databricks.
Both ensure discoverability, governance, and compliance.
-

Q22. Design disaster recovery for data infrastructure.

A22.

- Geo-replication for storage accounts.
- Multi-region cluster failover.
- Backup/restore for Synapse databases.
- Automated failover orchestration using Azure Site Recovery.

Q23. Use logging frameworks for traceability in data workflows.

A23.

- Use Python logging module in Spark jobs.
- Send structured logs to ELK/Datadog.
- Correlate logs with pipeline run IDs.
This aids debugging and compliance audits.

Q24. Implement data validation and quality checks.

A24.

- Schema validation.
- Null/constraint checks.
- Referential integrity validation.
- Aggregate reconciliation with source systems.
Bad records sent to quarantine tables for review.

Q25. Explain batch vs micro-batch scheduling design.

A25.

- **Batch:** Fixed intervals (e.g., daily).
- **Micro-batch:** Small intervals (e.g., every minute) via Spark Structured Streaming.
Choice depends on latency vs throughput trade-offs.

Q26. Guide evaluating RPO/RTO in pipeline SLAs.

A26.

- **RPO (Recovery Point Objective):** Max acceptable data loss (e.g., 15 mins).
- **RTO (Recovery Time Objective):** Max acceptable downtime (e.g., 30 mins).
Define SLAs, then architect pipelines to meet them.

Q27. Plan pipeline migration from on-prem to cloud.

A27.

- Assess existing workloads.

- Choose equivalent Azure services (SQL → Synapse, ETL → ADF).
 - Migrate schema/data using Data Migration Service.
 - Rebuild orchestration in ADF.
 - Test thoroughly before cutover.
-

Q28. Design multi-region data availability and replication.

A28.

- Store data in geo-redundant storage.
 - Use Azure Front Door for routing.
 - Sync metadata across regions.
 - Deploy processing clusters in active-active mode.
-

Q29. Explain snapshot vs incremental backup in data lakes.

A29.

- **Snapshot:** Full copy of data at a point in time. Higher cost, ensures consistency.
 - **Incremental:** Only changes since last backup. Cost-effective, but requires previous backups for restore.
-

Q30. Describe improving data pipeline CI/CD resilience.

A30.

- Add automated tests in build pipelines.
 - Use canary deployments.
 - Rollback strategies on failure.
 - Monitor deployment logs.
This minimizes risk during releases.
-

Amazon – Data Engineer Interview Q&A

Q1. Outline a data lake architecture using S3, Glue, and Athena.

A1.

- **Raw zone:** Land all source data in S3 (CSV, JSON, Parquet).

- **Cataloging:** Use Glue Data Catalog to define schema.
 - **Transformations:** Run ETL via Glue jobs or EMR.
 - **Curated zone:** Store cleaned/partitioned data.
 - **Analytics:** Query via Athena (serverless SQL).
- This architecture is scalable, low-cost, and central to AWS data engineering.
-

Q2. Design real-time analytics using Kinesis or Kafka streaming.

A2.

- **Producers** send events to Kinesis/Kafka.
 - **Consumers** (Lambda, Spark Streaming, Flink) process data.
 - Store in S3/Redshift for historical queries.
 - Use dashboards (QuickSight) for near real-time reporting.
- This supports fraud detection, clickstream analytics, or recommendations.
-

Q3. Handle schema evolution in Glue catalog datasets.

A3.

- Enable **schema versioning** in Glue.
 - Use **Parquet/Avro** formats supporting schema evolution.
 - Apply MERGE logic in Glue ETL jobs.
 - Add new columns as nullable to maintain backward compatibility.
-

Q4. Optimize Redshift queries for large datasets.

A4.

- Use **sort keys** and **distribution keys** wisely.
 - Vacuum and analyze tables periodically.
 - Compress data with column encoding.
 - Avoid cross-node shuffles by colocating joins.
 - Use **materialized views** for repeated queries.
-

Q5. Design a CDC system using Lambda and Kinesis.

A5.

- Capture changes from source DB via **DMS**.
 - Stream events into **Kinesis Data Streams**.
 - Process with Lambda/Flink for transformation.
 - Write changes to S3/Redshift.
This ensures low-latency replication and analytics.
-

Q6. Use S3 partitioning and file formats for performance.

A6.

- Partition by date, region, or category.
 - Store in **Parquet/ORC** for columnar efficiency.
 - Avoid too many small files; compact regularly.
This reduces scan cost in Athena/Redshift Spectrum.
-

Q7. Manage scheduler and orchestration via Step Functions or Airflow.

A7.

- **Step Functions**: Serverless orchestration with retries and state management.
 - **Airflow (MWAA)**: Complex DAGs with dependencies, Python-based flexibility.
Choice depends on workload complexity and team preference.
-

Q8. Build cost-cutting strategies for Spark on EMR.

A8.

- Use **spot instances** for worker nodes.
 - Enable **auto-scaling**.
 - Optimize Spark partitioning and caching.
 - Use S3 as storage (decouple from compute).
 - Terminate idle clusters automatically.
-

Q9. Monitor and alert using CloudWatch metrics.

A9.

- Track pipeline latency, failure count, throughput.
 - Configure alarms with SNS notifications.
 - Use custom metrics (row counts, job durations).
This ensures proactive incident handling.
-

Q10. Design disaster recovery for data pipelines.

A10.

- Enable cross-region replication of S3 buckets.
 - Backup Redshift snapshots.
 - Deploy multi-region EMR clusters.
 - Automate failover via Route 53 + Step Functions.
-

Q11. Explain encryption and IAM roles for secure data flows.

A11.

- Encrypt data **at rest** (KMS, SSE-S3) and **in transit** (TLS).
 - Use IAM roles with least-privilege access.
 - Rotate keys regularly.
 - Implement VPC endpoints for private S3 access.
-

Q12. Implement fault tolerance in streaming ingestion pipelines.

A12.

- Enable checkpointing in Spark/Flink.
 - Use multiple shards/partitions in Kinesis.
 - Retry failed events and route to **DLQ**.
 - Monitor consumer lag.
-

Q13. Describe in-lake aggregation strategies (like Presto layer).

A13.

- Store aggregated tables in curated S3 layer.
 - Use **Presto/Athena** for ad-hoc aggregations.
 - Precompute summary tables (daily revenue, active users).
This balances performance and storage cost.
-

Q14. Set up CI/CD using CodePipeline or Terraform for data infra.

A14.

- Use Terraform to define infrastructure (S3, Glue, Redshift).
 - Store scripts in Git.
 - Use CodePipeline to automate build, test, deploy.
 - Enable rollback on failure.
-

Q15. Balance batch vs stream processing tradeoffs.

A15.

- **Batch:** Efficient for large historical loads.
 - **Streaming:** Low latency, handles real-time data.
Amazon often uses **Kinesis for streaming + Glue/EMR for batch** in hybrid pipelines.
-

Q16. Handle large join optimizations: broadcast vs partitioned joins.

A16.

- **Broadcast join:** Small lookup table replicated to all nodes (fast).
 - **Partitioned join:** Large datasets shuffled by key.
Use broadcast for small dimensions, partitioned for big fact-to-fact joins.
-

Q17. Detect and clean corrupted data in ingestion.

A17.

- Validate schema using Glue crawler.
- Implement checksum/hash validation.
- Route invalid records to quarantine S3 bucket.

- Automate alerts for anomalies.
-

Q18. Use QuickSight for real-time dashboards.

A18. QuickSight connects to S3 (Athena), Redshift, or RDS.

- Create SPICE datasets for fast in-memory queries.
 - Build KPIs (active users, top-selling products).
 - Refresh data in near real-time using streaming sources.
-

Q19. Implement metadata catalog using Glue or Hive Metastore.

A19.

- **Glue Catalog:** Serverless, integrates with Athena/Redshift.
 - **Hive Metastore:** Works with EMR clusters.
Both provide schema discovery, versioning, and lineage.
-

Q20. Use S3 event triggers for pipeline orchestration.

A20. Example: When a new file lands in S3 → trigger Lambda → invoke Glue job or Step Function. This event-driven design eliminates polling and reduces latency.

Q21. Test pipelines locally using docker or emulators.

A21.

- Use **localstack** to emulate AWS services.
 - Run Glue/Spark scripts in Docker containers.
 - Mock S3/Kinesis streams for unit testing.
This enables fast feedback before deploying to AWS.
-

Q22. Incorporate data validation at pipeline boundaries.

A22.

- Validate schema and null constraints.
- Reconcile record counts between source and sink.
- Apply business rule checks (e.g., revenue > 0).
Invalid records sent to quarantine bucket.

Q23. Design data backups with lifecycle policies.

A23.

- Define S3 lifecycle policies (move to Glacier, then delete).
- Schedule Redshift snapshots.
- Store critical metadata in DynamoDB with point-in-time recovery.

Q24. Manage stale data retention with lifecycle rules or Glue.

A24.

- Set retention policy (e.g., delete raw data after 90 days).
- Use Glue jobs to archive data to Glacier.
- Maintain curated data for long-term analytics.

Q25. Explain AWS Lake Formation governance layers.

A25.

- Centralized permissions on Glue catalog tables.
 - Row/column-level security.
 - Integration with IAM and SSO.
- Lake Formation enforces fine-grained access for compliance.

Q26. Handle throttling and API limits in streaming sources.

A26.

- Use exponential backoff retries.
- Batch records before writes.
- Increase Kinesis shard count.
- Apply partition keys evenly to distribute load.

Q27. Orchestrate multi-region replication for analytics.

A27.

- Replicate S3 buckets using **Cross-Region Replication (CRR)**.

- Sync Glue catalogs across regions.
 - Use Redshift RA3 clusters with data sharing.
This supports global analytics.
-

Q28. Build A/B test ingestion pipelines and metrics.

A28.

- Split users randomly into groups.
 - Ingest clickstream events into S3.
 - Compute KPIs (CTR, dwell time).
 - Compare using statistical significance tests.
-

Q29. Secure data access with encryption and private VPC endpoints.

A29.

- Use **KMS for encryption**.
 - Restrict access with **S3 VPC endpoints**.
 - Configure **IAM roles** for least-privilege.
This avoids data exposure over the public internet.
-

Q30. Optimize query performance via proper partition keys in Athena/Redshift.

A30.

- Partition Athena tables by date or region.
 - Use Redshift **distribution keys** to colocate joins.
 - Apply column encoding and compression.
 - Run ANALYZE to refresh statistics.
-

Google – Data Engineer Interview Q&A

Q1. Outline BigQuery architecture—columnar storage, Dremel execution.

A1. BigQuery uses:

- **Columnar storage:** Data stored in compressed columnar format for fast scans.

- **Dremel execution engine:** Executes queries using a multi-level serving tree that parallelizes across nodes.
 - **Serverless model:** No cluster management; resources auto-scale. This makes queries extremely fast even on petabytes of data.
-

Q2. Optimize BigQuery performance—partitioning, clustering.

A2.

- Partition tables by date or ingestion_time.
 - Use clustering on frequently filtered columns.
 - Avoid SELECT *; select only needed fields.
 - Use **materialized views** for repeated aggregations.
 - Monitor queries with **Query Plan Explanation**.
-

Q3. Compare BigQuery vs Redshift tradeoffs.

A3.

- **BigQuery:** Serverless, pay-per-query, elastic scaling. Ideal for ad-hoc workloads.
 - **Redshift:** Provisioned clusters, predictable cost, more tuning required. Better for consistent, long-running queries.
- Google often prefers BigQuery for flexibility and scale.
-

Q4. Manage schema evolution in BigQuery tables.

A4.

- Add new nullable columns (backward compatible).
 - Use **ALTER TABLE** for schema changes.
 - Store schema versions in metadata tables.
 - For breaking changes, write to new table + migrate gradually.
-

Q5. Explain Dataflow (Apache Beam) pipeline design.

A5.

- **Input:** Read from Pub/Sub, BigQuery, or GCS.

- **Transform:** Apply PCollection transformations (Map, GroupByKey, Windowing).
 - **Output:** Write to sinks like BigQuery or GCS.
 - **Autoscaling** ensures cost efficiency.
Apache Beam model allows portability across engines (Dataflow, Flink, Spark).
-

Q6. Implement streaming vs batch pipelines in Beam.

A6.

- **Batch:** Fixed dataset, bounded PCollections. Example: historical log processing.
 - **Streaming:** Unbounded PCollections with windowing, watermarking, triggers.
Example: real-time user activity analytics.
Beam unifies both modes in a single programming model.
-

Q7. Use Pub/Sub for ingestion and scaling.

A7.

- Pub/Sub supports global, horizontally scalable messaging.
 - Producers publish events to topics; consumers subscribe.
 - Supports **at-least-once delivery**.
 - Integrated with Dataflow for real-time ingestion.
-

Q8. Apply Data Studio dashboards on BigQuery.

A8.

- Connect Data Studio directly to BigQuery tables.
 - Create parameterized dashboards with filters (date, region).
 - Use BI Engine for in-memory acceleration.
This enables interactive analytics on BigQuery data.
-

Q9. Implement Airflow on GCP with Cloud Composer.

A9.

- Cloud Composer is managed Airflow.
- Define DAGs in Python (ETL workflows).

- Integrate with BigQuery, GCS, Dataflow operators.
 - Supports monitoring and retry policies.
This simplifies orchestration of complex workflows.
-

Q10. Discuss distributed joins and shuffling in Dataflow.

A10.

- Joins require shuffling data by key across workers.
 - Expensive for large datasets; optimize with **side inputs** for small tables.
 - Use **partitioning** to minimize shuffle cost.
-

Q11. Handle late data with watermarking and triggers.

A11.

- **Watermark:** Defines event-time progress.
 - **Triggers:** Decide when to emit results (on-time, late, after certain count).
Example: Window user clicks by minute with 5-min late tolerance.
-

Q12. Implement deduplication in streaming pipelines.

A12.

- Use **windowing + keyBy** in Beam to group duplicates.
 - Apply distinct transform or use stateful processing.
 - Maintain deduplication keys in external store (e.g., Bigtable).
-

Q13. Compare Bigtable vs BigQuery use cases.

A13.

- **Bigtable:** Low-latency, NoSQL, time-series data (IoT, personalization).
 - **BigQuery:** OLAP, ad-hoc queries on large datasets.
Both complement each other in Google-scale systems.
-

Q14. Write SQL to compute top N per group in BigQuery.

A14.

```
SELECT user_id, movie_id, rating
FROM (
  SELECT user_id, movie_id, rating,
         ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY rating DESC) AS rnk
  FROM ratings
)
WHERE rnk <= 3;
```

Q15. Use ARRAY and STRUCT types in BigQuery.
A15.

```
SELECT
  user_id,
  ARRAY_AGG(STRUCT(movie_id, rating) ORDER BY rating DESC LIMIT 3) AS top_movies
FROM ratings
GROUP BY user_id;
```

This nests results into arrays of structs per user.

Q16. Apply partition pruning in queries.

A16. Ensure queries include **partition filter**:

```
SELECT *
FROM sales
WHERE _PARTITIONDATE BETWEEN '2024-01-01' AND '2024-01-31';
```

This avoids full table scans.

Q17. Write efficient queries avoiding cross joins.

A17.

- Use explicit join conditions.
- For cartesian product, ensure downstream filters reduce rows.
- Replace with window functions where possible.

Q18. Debug slow queries in BigQuery via EXPLAIN.**A18.**

- Check slots used and shuffle stages.
- Look for excessive scans (due to no partition filter).
- Optimize with clustering, partitioning, or materialized views.

Q19. Design ML pipeline with BigQuery ML.**A19.**

- Train directly in BigQuery:

```
CREATE OR REPLACE MODEL my_model OPTIONS(model_type='linear_reg') AS
```

```
SELECT features, label FROM training_data;
```

- Evaluate with ML.EVALUATE.
 - Predict with ML.PREDICT.
- This allows analysts to build models using SQL.

Q20. Handle nested JSON ingestion into BigQuery.**A20.**

- Use STRUCT and ARRAY data types.
- Ingest JSON via Dataflow or native JSON loader.
- Flatten with UNNEST() during queries.

Q21. Secure data with IAM and VPC SC in GCP.**A21.**

- IAM roles: BigQuery Admin, Viewer, etc.
- VPC Service Controls: Prevent data exfiltration.
- CMEK (Customer-managed keys) for encryption.

Q22. Monitor BigQuery cost with budget alerts.**A22.**

- Set budgets in Cloud Billing.
 - Track query costs via INFORMATION_SCHEMA.
 - Use slot reservations for predictable pricing.
-

Q23. Optimize storage cost with table lifecycle policies.

A23.

- Set table expiration (e.g., 90 days).
 - Move cold data to GCS.
 - Use clustered tables to reduce query cost.
-

Q24. Design multi-region pipelines in GCP.

A24.

- Store data in multi-region GCS buckets.
 - Replicate BigQuery datasets across regions.
 - Use Pub/Sub global topics for ingestion.
-

Q25. Implement CI/CD for Dataflow jobs with Cloud Build.

A25.

- Store pipeline code in Git.
 - Use Cloud Build triggers for commits.
 - Deploy Dataflow templates automatically.
 - Validate with unit tests + integration tests.
-

Q26. Compare Cloud Spanner vs BigQuery.

A26.

- **Cloud Spanner:** OLTP, strongly consistent, horizontally scalable relational DB.
 - **BigQuery:** OLAP, analytical workloads.
Use Spanner for transactions, BigQuery for analytics.
-

Q27. Handle GDPR/CCPA compliance in GCP pipelines.

A27.

- Use DLP API to mask/anonymize PII.
 - Implement retention policies.
 - Enable audit logging.
 - Restrict access with IAM.
-

Q28. Manage data lineage in GCP workflows.

A28.

- Use **Data Catalog** for metadata.
 - Integrate with **Dataplex** for governance.
 - Enable audit logs for transformations.
- Lineage is essential for trust and compliance.
-

Q29. Debug pipeline failures in Dataflow.

A29.

- Inspect logs in Stackdriver.
 - Check worker resource usage.
 - Verify watermark progression for streaming.
 - Retry failed jobs with checkpoint recovery.
-

Q30. Explain Google's culture fit for Data Engineer role.

A30. Google values **innovation, collaboration, and impact at scale**. A DE should:

- Think about building systems for billions of users.
- Prioritize automation and scalability.
- Communicate clearly across teams.
- Balance technical excellence with business impact.

Q1. Design a retail data warehouse schema for sales.

A1.

- **Fact Table:** sales_fact (sale_id, date_id, store_id, product_id, customer_id, quantity, revenue).
 - **Dimensions:** date_dim, store_dim, product_dim, customer_dim.
 - Partition fact table by date_id.
This **star schema** supports fast queries for KPIs like daily sales and category performance.
-

Q2. Optimize ETL pipeline for nightly sales data loads.

A2.

- Use **incremental loads** (load only new/changed records).
 - Parallelize ingestion (by region or date).
 - Compress & partition files (Parquet).
 - Schedule pipeline with Airflow/ADF.
 - Monitor using metrics (row counts, latency).
-

Q3. Handle slowly changing dimensions for product catalog.

A3.

- **Type 1:** Overwrite for corrections.
 - **Type 2:** Add new row with start/end date for history (e.g., price changes).
 - **Type 3:** Keep previous value in extra column (limited history).
Walmart uses SCD Type 2 for tracking product price/version history.
-

Q4. Manage schema evolution in product and inventory datasets.

A4.

- Use Delta Lake or Hive with **schema evolution enabled**.
- Add new columns as nullable.
- Maintain metadata in Glue/Hive metastore.
- Alert downstream consumers of changes.

Q5. Handle late-arriving sales data.

A5.

- Use **watermarking** in streaming jobs.
- Store late records in staging → merge into fact table.
- Maintain audit logs for reconciliation.

Q6. Write SQL for top-selling products by region.

A6.

```
SELECT region, product_id, SUM(quantity) AS total_qty
FROM sales_fact s
JOIN store_dim st ON s.store_id = st.store_id
GROUP BY region, product_id
ORDER BY region, total_qty DESC;
```

Q7. Query customers with no purchases in past 6 months.

A7.

```
SELECT c.customer_id, c.name
FROM customer_dim c
LEFT JOIN sales_fact s
  ON c.customer_id = s.customer_id
  AND s.sale_date >= DATEADD(MONTH, -6, CURRENT_DATE)
WHERE s.customer_id IS NULL;
```

Q8. Compute basket size distribution across stores.

A8.

```
SELECT store_id, AVG(items_per_basket) AS avg_basket_size
FROM (
  SELECT sale_id, store_id, COUNT(product_id) AS items_per_basket
```

```
FROM sales_fact

GROUP BY sale_id, store_id

)t

GROUP BY store_id;
```

Q9. Compare Snowflake vs Redshift vs BigQuery for Walmart.

A9.

- **Snowflake:** Elastic scaling, easy multi-cloud.
 - **Redshift:** Tight AWS integration, predictable cost.
 - **BigQuery:** Serverless, best for ad-hoc analysis.
Choice depends on workload type and corporate cloud strategy.
-

Q10. Optimize inventory restock prediction pipelines.

A10.

- Use **streaming ingestion** of sales.
 - Train ML models for demand forecasting.
 - Aggregate sales by region, season, product.
 - Automate restock alerts via dashboards.
-

Q11. Ensure data quality in sales pipelines.

A11.

- Apply validation rules (no negative sales).
 - Reconcile counts with source systems.
 - Store invalid rows in quarantine tables.
 - Automate data profiling checks.
-

Q12. Secure customer PII in data warehouses.

A12.

- Encrypt at rest and in transit.

- Tokenize or mask PII (phone, email).
 - Restrict access via RBAC.
 - Enable column-level security in warehouse.
-

Q13. Handle streaming sales ingestion with Kafka.

A13.

- Producers = POS systems publishing events.
 - Kafka topics store sales events.
 - Consumers (Spark/Flink) process → S3/Delta Lake.
 - Aggregate in real-time for dashboards.
-

Q14. Manage pipeline orchestration at Walmart scale.

A14.

- Use **Airflow/Step Functions** for orchestration.
 - DAGs handle dependencies across sales, inventory, HR data.
 - Implement retries + SLA monitoring.
-

Q15. Optimize Spark jobs for terabytes of POS data.

A15.

- Repartition on store_id or date.
 - Cache reused datasets.
 - Use broadcast joins for small lookup tables.
 - Monitor DAG in Spark UI.
-

Q16. Apply machine learning for personalized promotions.

A16.

- Build recommendation models (ALS, collaborative filtering).
- Use customer purchase history.

- Deliver promotions via mobile apps/email.
This increases upselling and loyalty.
-

Q17. Handle GDPR/CCPA compliance for customer data.

A17.

- Implement “right to be forgotten” workflows.
 - Use pseudonymization of identifiers.
 - Maintain consent metadata.
 - Audit access logs regularly.
-

Q18. Design multi-region replication for data warehouses.

A18.

- Replicate fact/dimension tables to each region.
 - Use CDC for syncing changes.
 - Deploy read replicas closer to analysts.
This ensures availability and low latency globally.
-

Q19. Manage metadata with data catalogs.

A19.

- Use Glue Catalog or Collibra for metadata.
 - Track lineage from ingestion → warehouse → BI.
 - Enable business glossary for consistency.
-

Q20. Implement CI/CD for ETL pipelines.

A20.

- Store pipeline code in Git.
 - Validate via Jenkins/Azure DevOps.
 - Deploy via Terraform/CloudFormation.
 - Promote Dev → QA → Prod with approvals.
-

Q21. Troubleshoot pipeline delays in peak sales.

A21.

- Check source system lag.
 - Verify Kafka consumer lag.
 - Scale clusters via autoscaling.
 - Reprocess backlogs from checkpoint.
-

Q22. Write SQL for average spend per customer.

A22.

```
SELECT customer_id, AVG(total_amount) AS avg_spend
FROM (
  SELECT customer_id, SUM(revenue) AS total_amount
  FROM sales_fact
  GROUP BY sale_id, customer_id
) t
GROUP BY customer_id;
```

Q23. Debug missing sales data in Redshift/Snowflake loads.

A23.

- Compare source vs target row counts.
 - Check ingestion logs for failed batches.
 - Inspect staging tables.
 - Reprocess missed data.
-

Q24. Monitor pipeline SLAs in production.

A24.

- Track job completion times.
- Alert on SLA breaches.
- Use Grafana dashboards + CloudWatch.

- Escalate critical failures automatically.
-

Q25. Implement anomaly detection in sales data.

A25.

- Use statistical models (z-score) for outlier sales.
 - Train ML models on seasonal patterns.
 - Flag anomalies in dashboards.
-

Q26. Manage cost in large-scale cloud pipelines.

A26.

- Compress data (Parquet, ORC).
 - Use autoscaling clusters.
 - Apply lifecycle policies for cold data.
 - Monitor spend with budget alerts.
-

Q27. Write Python for deduplication in ETL.

A27.

```
df = df.dropDuplicates(["customer_id", "sale_id"])
```

Ensures only unique records are loaded.

Q28. Align pipeline reliability with Walmart's 24/7 operations.

A28.

- Build **HA clusters**.
 - Monitor pipelines continuously.
 - Ensure recovery via checkpointing.
 - Scale resources dynamically during holidays/Black Friday.
-

Q29. Integrate IoT/sensor data with sales analytics.

A29.

- Ingest IoT data (inventory sensors) via Kafka.
 - Merge with sales transactions.
 - Use analytics to track real-time shelf stock.
This reduces stockouts and improves supply chain efficiency.
-

Q30. Fit Walmart's culture of scale, cost efficiency, and reliability.

A30. Walmart values **low-cost, high-scale solutions**. For a DE, this means:

- Designing pipelines that scale to billions of records.
- Monitoring costs closely.
- Prioritizing uptime and customer trust.

Atlassian – Data Engineer Interview Q&A (30)

1. **How would you ingest and process over 1 million events per day efficiently?**
Answer: Use distributed ingestion (e.g., Kafka or Kinesis), process in Spark with partitioning, store in performant formats like Parquet with compression. Ensure parallelism, reliability, and scalable data storage. [Medium](#)
2. **Design a data model for impressions and clicks from marketing campaigns.**
Answer: Separate fact tables for impressions and clicks with event IDs, timestamps, campaign/user IDs; joinable dimensions; use a star schema for analytics. [Medium](#)
3. **How do you handle late-arriving data and deduplication?**
Answer: Use watermarking in Spark to allow delay windows, dedupe via ROW_NUMBER() over keys, and enforce validations in ingestion layer. [Medium](#)
4. **Write SQL to calculate average bug resolution time per team.**
Answer: Joining bugs and teams, compute resolved_at - created_at, group by team, and aggregate average resolution time. [DataLemur](#)
5. **SQL: Find the most frequently used Atlassian product last month.**
Answer: Group usage by product name, count, filter by date, then order by count descending and LIMIT 1. [DataLemur](#)
6. **SQL: Rolling average or sum over the last 3 days.**
Answer: Use window functions with OVER (ORDER BY date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW). [Glassdoor](#)

7. **Python: Search for target number position in an array.**
Answer: Simple loop approach; alternative: binary search for sorted arrays.
[Glassdoor](#)
8. **Explain Atlassian's coding interviews (data structures & code design).**
Answer: Interview assesses data structures and code design using your preferred language; focus on thought process, trade-offs, and clarity—not perfection. [Atlassian](#)
9. **Behavioral fit: "Open company, no BS" culture example.**
Answer: Describe a time when honest, transparent communication resolved a project hurdle, driving better outcomes. [AlgoMonster](#)
10. **Why Atlassian? Favorite Atlassian product? (Behavioral)**
Answer: Speak from experience—highlight tool depth (Jira/Confluence), ease of use, and how it aligns with your workflow and values. [Exponent](#)
11. **System design: Build a real-time, scalable data pipeline.**
Answer: Ingest via Kafka → Spark Streaming → store raw & curated in Data Lake → serve via Snowflake or BI tools; add monitoring & CI/CD. [Interview Query](#)
12. **SQL: First-touch attribution channel per user.**
Answer: Use MIN(created_at) per user joining attribution and session tables to extract earliest touch channel. [Interview Query](#)
13. **System design: Predict optimal commercial break in livestream video.**
Answer: Use signal processing, content markers, user engagement metrics, build ML model to predict breakpoints, integrate in event pipelines. [Interview Query](#)
14. **How do you ensure data quality in pipelines?**
Answer: Use validation frameworks (e.g., Great Expectations), null checks, schema contracts, DLQs for anomalies, and reconciliation checks. [Interview Query](#)
15. **CI/CD: Deploy Spark jobs and data pipelines.**
Answer: Use Git + Bitbucket, define pipelines, use CI for validation, deploy via CI/CD pipelines with approvals. [Interview Query](#)
16. **Storytelling: Share a time you failed and learned in pipeline delivery.**
Answer: Use STAR method—Situation, Task, Action, Result; highlight ownership, resolution, learning, and process improvement.
17. **Explain your experience building pipelines on cloud-native tools (Snowflake, Databricks).**

Answer: Discuss using Databricks for ETL, applying data models in Snowflake, and enabling data access for analytics. [Interview Query](#)

18. What monitoring tools do you use in production pipelines?

Answer: Use Spark UI, custom dashboards, logging frameworks, and alerting tools for full pipeline visibility.

19. How do you model a “teamwork graph” for analytics like Rovo?

Answer: Represent nodes as users/projects, edges with interaction weights, store graph data in property graph DB or graph analytics engine.

20. How do you maintain schema evolution in data warehouses?

Answer: Use schema-on-write formats (Delta), versioning, backfill processes, and consumer coordination for schema changes.

21. How do you optimize Spark jobs at scale?

Answer: Partition data appropriately, broadcast small lookups, cache repeatedly used data, and monitor shuffles/skew.

22. Behavioral: Working with cross-functional teams remotely.

Answer: Emphasize clear documentation, async tools (Confluence), shared roadmaps, and empathy in communication.

23. Question: Select top 3 departments with ≥ 10 employees earning $> 100K$.

Answer: Use CTEs, GROUP BY, HAVING COUNT ≥ 10 , compute percentage using conditional AVG, ORDER BY percentage DESC, LIMIT 3. [Interview Query](#)

24. Behavioral: Mentoring junior engineers.

Answer: Highlight how you've paced onboarding, pair-programmed, and helped grow skills and culture.

25. What's Atlassian's interview loop structure?

Answer: Typically SQL / coding, design, values, and hiring bar review. Fast feedback and focus on impact. [Interview Query](#)

26. How do you communicate technical work to non-tech stakeholders?

Answer: Use metrics, dashboards, simplify trade-offs, and tie outcomes to business goals.

27. Explain how to calculate rolling counts (e.g., errors per 3-day window).

Answer: Use window functions (SUM() OVER ordered rows RANGE or ROWS).

28. How to handle ambiguity in requirements?

Answer: Ask clarifying questions, state assumptions, propose incremental plan, confirm next steps.

29. What would be your proudest pipeline improvement?

Answer: Describe a performance boost, reliability improvement, or reduction in MTTR (mean time to recover).

30. Advice on culture fit for Atlassian?

Answer: Embrace transparency, ownership, empathy, bias for action, and continuous improvements.

Tesla – Data Engineer Interview Questions & Answers (30)

SQL & Data Modeling

Q1. Write SQL to select the top 3 departments with at least 10 employees and highest average salary.

A1.

```
WITH dept_stats AS (  
    SELECT department, COUNT(*) AS emp_count, AVG(salary) AS avg_sal  
    FROM employees  
    GROUP BY department  
)  
SELECT department  
FROM dept_stats  
WHERE emp_count >= 10  
ORDER BY avg_sal DESC  
LIMIT 3;
```

Q2. How would you find the first-touch attribution channel for each user signup?

A2. Use ROW_NUMBER() partitioned by user and ordered by event timestamp. Select the earliest channel per user.

Q3. Write a query to count users who made a purchase after an upsell event.

A3.

```
SELECT COUNT(DISTINCT u.user_id)
```

```
FROM events e1  
JOIN events e2 ON e1.user_id = e2.user_id  
WHERE e1.event_type = 'upsell'  
AND e2.event_type = 'purchase'  
AND e2.timestamp > e1.timestamp;
```

Q4. How do you calculate the average number of swipes per user per day?

A4. Divide total swipes by distinct active days per user.

```
SELECT user_id, COUNT(*) / COUNT(DISTINCT DATE(event_time)) AS avg_swipes  
FROM swipes  
GROUP BY user_id;
```

Q5. How would you retrieve the latest salary per employee and detect mismatches?

A5. Use a window function with ROW_NUMBER() over ORDER BY timestamp DESC to get the most recent record. Compare against HR or payroll reference values.

Q6. How do you group sequential timestamps into weekly buckets starting from the first timestamp?

A6. Use Python: iterate through sorted timestamps, group into 7-day intervals, and handle edge cases with timezone normalization.

Q7. Write SQL to calculate total charging time per Tesla station and compare with the previous day.

A7. Use aggregation per day, then LAG() window function to compare against prior day totals.

Q8. Write SQL to identify parts that have been started but not finished in the assembly line.

A8.

```
SELECT part_id  
FROM assembly
```

WHERE start_time IS NOT NULL AND end_time IS NULL;

Q9. Explain the purpose of the CHECK constraint in SQL.

A9. Ensures column data meets conditions, e.g., CHECK (salary > 0). Used for enforcing business rules at the database level.

Q10. Write SQL to calculate ad click-to-cart conversion rate by campaign and product.

A10. Use two aggregated CTEs: one for clicks, one for add-to-carts, then join to calculate the ratio.

Python & Algorithms

Q11. Write Python code to return all bigrams from a sentence.

A11.

```
tokens = sentence.split()
```

```
bigrams = [(tokens[i], tokens[i+1]) for i in range(len(tokens)-1)]
```

Q12. How do you stem words using a dictionary of roots?

A12. Build a **trie** of root words. For each word, traverse until shortest matching root is found, then replace it.

Q13. How do you find the top N frequent words from a text block?

A13. Tokenize, count with a dictionary or collections.Counter, then extract top N using heap or sorting.

Q14. How do you compute the angle between the hour and minute hands of a clock?

A14. Formula:

$\text{abs}(30 \cdot \text{hour} - 5.5 \cdot \text{minutes}) \rightarrow \text{if } > 180, \text{ subtract from } 360.$

Q15. Explain merge sort, quicksort, and heap sort.

A15.

- Merge sort: divide & merge, $O(n \log n)$.
 - Quick sort: partition & recurse, average $O(n \log n)$.
 - Heap sort: build heap, extract max/min iteratively, $O(n \log n)$.
-

Q16. Explain dynamic programming with the Longest Common Subsequence (LCS) example.

A16. Use a DP table of size $(m+1) \times (n+1)$. If characters match, increment diagonal; else, take max of left/top cell.

Q17. Describe Dijkstra's and Prim's algorithms.

A17.

- **Dijkstra's:** shortest path from a source using priority queue.
 - **Prim's:** minimum spanning tree, greedy selection of edges.
-

Q18. How would you evaluate a set of equations with dependencies?

A18. Represent as a dependency graph, perform **topological sort**, evaluate nodes in order.

System Design

Q19. How would you design a system to find the shortest path between two Tesla Superchargers?

A19. Model chargers as graph nodes, roads as edges with weights (distance/time). Apply Dijkstra or A* algorithm.

Q20. Design a schema for retail receipts that include coupons.

A20.

- **Receipts:** receipt_id, date, customer_id.
- **Items:** item_id, product_id, receipt_id, price.

- **Coupons:** coupon_id, discount_value.
 - **Receipt_Coupons:** mapping between receipts and coupons.
-

Q21. Explain what happens when you type google.com in a browser.

A21. Steps: DNS lookup → TCP handshake → TLS negotiation → HTTP request → server response → rendering engine displays content.

Q22. How would you design a task scheduler with dependencies?

A22. Represent tasks as a DAG, use topological sort to execute respecting dependencies. Retry on failures.

Tesla-Specific Use Cases

Q23. Write SQL to calculate average distance and power consumed per Tesla model and year.

A23.

```
SELECT v.model_name, v.manufacture_year,  
       AVG(s.distance_driven) AS avg_distance,  
       AVG(s.power_consumed) AS avg_power  
FROM vehicles v  
JOIN service_records s ON v.vehicle_id = s.vehicle_id  
GROUP BY v.model_name, v.manufacture_year;
```

Q24. How would you build a pipeline for vehicle telemetry data?

A24. Ingest data via Kafka → process in Spark Streaming (window functions for metrics) → store in Delta Lake → expose via dashboards.

Q25. How do you ensure data quality in manufacturing pipelines?

A25. Apply validation checks (nulls, ranges), reconcile counts across stages, quarantine invalid records, set up monitoring alerts.

Behavioral & Culture Fit

Q26. What was the toughest professional problem you solved?

A26. Use STAR method. Example: “Diagnosed Spark shuffle bottlenecks in ETL, repartitioned data, reduced runtime by 40%.”

Q27. Tell me about a time you failed.

A27. Acknowledge mistake, explain corrective actions, highlight learning. Tesla values resilience.

Q28. Why do you want to work at Tesla?

A28. Express passion for Tesla’s mission (sustainable energy, innovation) and show alignment with first-principles thinking.

Q29. What’s your favorite Tesla product and why?

A29. Example: Model Y for efficiency, range, and scalability in mass market—connect to company mission.

Q30. Tell me about a disagreement with your manager and how you resolved it.

A30. Use STAR: respectful disagreement, present evidence, align on goals. Tesla values boldness + teamwork.
